

CPT204 Data Structures and Algorithms - 24-25F

Part A. Gap-fill Questions

1. __ allows different classes to define methods with same name but behave differently, while __ ensure that the internal class data is not directly accessible from outside the class;

sol: Polymorphism, Encapsulation

2. __ store objects that are processed in a last-in, first-out fashion, while __ store objects that are processed in a first-in, first-out fashion.

sol: Stack, Queue

3. Given the Following method call:

```
public static void update(int num, int[] array) {  
    num = 100;  
    array[0] = 50;  
}
```

what is the printed value of x in the main method?

```
int x = 10; int[] arr = new int[1];  
update(x, arr);  
System.out.println("x = " + x);
```

sol:

because num is a primitive type, it is passed by value (copy), so the change to num does not affect x. but array is a reference type, it is passed by reference (address), so the change to array[0] does affect arr[0]. Therefore, the output will be: x = 10

4. What is the default value of myArray[2] after this declaration?

```
boolean[] myArray = new boolean[5];
```

sol:

the element in the array are boolean type will be initialize to false by default.

5. What is the output of the following code

```
String s1 = "Hello";
String s2 = new String("Hello");
System.out.println(s1 == s2);
```

sol:

String s1 = " Hello" ; creates a string literal stored in the string pool in the compiling period. String s2 = new String(" Hello"); create a new String object in the heap. the address of s1 and s2 are different, so the output will be false.

6. Given the Student class below:

```
public class Student {
    private List<String> courses;
    public List<String> getCourses() {
        return courses;
    }
}
```

Although course is private, the class is still not immutable because the getter returns a/an ___ to mutable object.

Sol:

reference the getter method getCourses() returns a reference to the mutable List<String> object, which allows external code to modify the contents of the courses list, thus making the Student class not immutable.

7.

```
public boolean equals(Object o) {
    if (o instanceof Circle) {
        return radius == ((Circle) o).radius;
    }
    return false;
}
```

After ___, you can access the field. Equality must be based on comparing relevant fields;

sol: casting

8.

```
class B {
    public void p(double i) {
        System.out.println(i * 2);
    }
}
class A extends B {
    public void p(int i) {
        System.out.println(i);
    }
}
```

```

    }
}

```

Given that method overloading is based on the reference type at compile time, what is printed by

```

B a = new A();
a.p(10);

```

sol:

The variable `a` has static type `B`, but its dynamic type is `A`. Method overloading is resolved at compile time according to the static type of the reference. Since `a` is declared as type `B`, the compiler only considers methods available in class `B`. Class `B` has method `p(double)`. Therefore, the argument `10` is widened from `int` to `double`, and the method call is resolved as:

```

B.p(double)

```

At runtime, dynamic dispatch only applies to overriding methods. However, class `A` does not override `p(double)`; it only overloads `p(int)`. Therefore, `B.p(double)` is executed. 左边类型决定能调用哪些方法。右边对象决定 override 后实际执行哪个版本。overload 看左边, override 看右边。
20.0

9:

In the code below:

```

interface T1 {
    int K = 1;
}
System.out.println(T1.K);

```

What kind of static member –which is fixed and cannot be changed –is accessed using the interface name `T1`?

sol:

this means, what is decorating the field `K` in the interface `T1`? the default modifier for field in an interface is `public static final`, so `K` is a constant field, which is fixed and cannot be changed. `final` (constant)

10:

```

House house1 = new House(1, 1750.50);
House house2 = ( House ) (house1.clone())
System.out.println(house1.getWhenBuilt() == house2.getWhenBuilt());

```

the code above performs a / an ____ if `clone()` is not overridden;

sol:

shallow copy the default clone() method is Object class performs a shallow copy by operation like field-by-field copy, where some reference fields are copied as references, not the objects they point to.

11

if the key is the first element in the array, the linear search completes in __ comparisons.

sol:

2 if you want to count the comparison in for(i = 0; i < arr.length; i++) and if(arr[i] == key) 1 if you only count the comparison in if(arr[i] == key) (I guess this may be more reasonable) the linear search code

```
static int linearSearch(int[] arr, int key) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == key) {
            return i;
        }
    }
    return -1;
}
```

the comparisons happen when

1. i < arr.length is true.
2. arr[i] == key

12

In the class, we introduced various methods for recording OOP learning, among which as __ is a means of recording ideas, personal thought and experiences, as well as insight you gain in the learning process;

sol:

Learning Journal?

13:

We have learned several different data structures in Java, Suppose you are asked to design a container that can support efficient and random access through an index, and efficient insertion and deletion from the end. The best data structure for the container is __;

sol:

ArrayList is more suitable for this scenario

1. ArrayList provides efficient random access.
2. although insertion and deletion from the middle or head of ArrayList requires shifting elements, insertion and deletion from the end of ArrayList is efficient with Complexity O(1)

14.

We have learned several different data structures in Java. Suppose you are asked to design a container that can support efficient and random access through an index; in addition, it supports insertion and deletion from the end by multiple threads concurrently. The best data structure for the container is __

sol:

Vector

15.

What is the output of the following code segment

```
PriorityQueue<String> queue = new PriorityQueue<String>(
    Arrays.asList("Liverpool", "Suzhou", "London", "Beijing"));
while (!queue.isEmpty())
    System.out.print(queue.poll() + " ");
```

sol:

the priority queue is sorted according to the alphabetical order, if the first letter are same, then compare the second letter and so on, So the output will be Beijing, Liverpool, London, Suzhou

16.

A method called “method1” for the CPTList class is shown below. Assume there are two CPTList objects, list1 and list2. list1 contains 3 nodes of value 1, 2, and 3 (starting from the front). list2 contains another 3 nodes of value 4, 5, and 6 (starting from the front). What would be the nodes in list1 after list1.method1(list2)? Place a comma between neighboring nodes in your answer, e.g. 1,3,2,4 (although comma is not a part of the linked list data structure).

```
public class CPTListNode {
    int data;
    CPTListNode next;
}
public class CPTList {
    private CPTListNode front;
    // some methods
    public void method1(CPTList other){
        CPTListNode I = other.front;
        while (I.next != null){
            I = I.next;
        }

        I.next = front;
        front = other.front;
    }
}
```

sol:

original list1 1 -> 2 -> 3 -> null list1.front = 1 list2 4 -> 5 -> 6 -> null list2.front = 4 in the method1 after while, I = 6, I.next = this.front => 4 -> 5 -> 6 -> 1 -> 2 -> 3 -> null this.front = 1, other.front

= 4 this.front = other.front => no change, only change the reference of list. the node link is not changed (only node.next = operation will change the link) so 4, 5, 6, 1, 2, 3

17.

A Map is a special type of Java container that stores a collection of key/value pairs. We have learned 3 types of built-in maps. They are: , and

sol:

1. HashMap —基于哈希表，无序， $O(1)$ 操作
2. LinkedHashMap —按插入顺序迭代
3. TreeMap —按键的自然顺序（或 Comparator）排序，红黑树实现， $O(\log n)$

18.

Suppose we have 3 containers below each containing the same set of students IDs at xjtlu. container1 is a LinkedHashSet. container2 is a LinkedList. container3 is an ArrayList. The student IDs are stored in a random order in all 3 containers. Now We want to check if a given id, say 2209999, exists in the containers using the method “contains()”. The code segment would be like

```
container1.contains(2209999)
container2.contains(2209999)
container3.contains(2209999)
```

Which container is likely to have the minimum time consumption?

sol:

- container1 is a LinkedHashSet. the contains method is implemented using its super class (HashSet), which is based on hash table to search the element, the Complexity is $O(1)$ on average.
- container2 is a LinkedList. the contains method is implemented by iterating through the linked list, and comparing each element with the given id. the Complexity is $O(n)$
- container3 is an ArrayList. the arraylist uses iteration to search the element, the Complexity is $O(n)$, too

19.

In Big O notation, the term __ is used to describe algorithms with complexity that grow at a linear rate with the input size.

sol:

linear time complexity

20

The worst-case time complexity for selection sort, insertion sort, bubble sort, and quick sort algorithms is __

sol:

asshole quicksort.

- selection sort: select the smallest element from the unsorted part and swap it with the first element of unsorted part. the worst-case time complexity is $O(n^2)$
- Insertion sort:
 1. initialize the list into unsorted and sorted part (only one element), the first element is in the sorted part.
 2. iteratively take the first element from the unsorted part, and insert it in to the sorted part's correct position. (compare with the last element in the sorted part, if it is bigger than the last element, then insert it after the last element.)
 3. worst case time complexity is $O(n^2)$, while the best case time complexity is $O(n)$
- bubble sort: repeatedly swap the adjacent element if they are in the wrong order, the worst-case time complexity is $O(n^2)$, while the best case time complexity is $O(n)$ when the list is already sorted.
- quick sort:
 1. select a pivot element from the list, and partition the list into two sub-lists: one with value less than the pivot, and the other with value greater than the pivot.
 1. recursively apply the above steps to the sub-lists until the base case is reached (when the list has one or zero elements).
 2. worst-case time complexity is $O(n^2)$ when the smallest or largest element is always chosen as the pivot, while the average and best case time complexity is $O(n \log n)$ when the pivot divides the list into two equal halves.

21:

```
int count = 0;
for (int i = 1; i < n; i *= 2) {
    count++;
}
```

What is the time complexity of the code above?

sol:

$O(\log(n))$ analysis: each time the loop will increase the i by multiplying 2 the value of i will be 1, 2, 4, 8, 16, ..., 2^k when $i < n$, the loop will continue, so we have $2^k < n$ count = $k = \log_2(n)$

22.

In Merge Sort, the array is repeatedly divided into smaller sublists until each sublist contains only __ element.

sol:

1

23. What value should best replace the ‘?’ to maintain the max heap strictly?

12 - ?, 10 ? - 8, 9

sol: 11?

10 - no children in the Max heap, the parent node should be greater than its children, so $12 \geq ? \geq 9$ therefore, the possible value for ? is $12 - 9$ but for "Strictly max heap" : 11

24

The time complexity for inserting an element into, and deleting an element from, a binary search tree are __ and __ respectively, assuming that the tree is balanced.

sol:

- inserting and deleting element from a balanced binary search tree is $O(\log n)$

but for unbalanced binary search tree, the worst complexity is $O(n)$ cause it may become a linked list and we have to use linear search to find the element to insert or delete;

25.

What does Dijkstra's algorithm compute?

sol:

weight graph without negative weight edges Single Source Shortest Path Problem

26.

In an AVL tree, a __ imbalance occurs when a node is inserted into the right subtree of the right child of A, while a __ imbalance occurs when a node is inserted into the left subtree of the left child of A.

sol:

RR, LL

27.

The postorder traversal of the elements in an AVL tree after inserting 15, 10, 20, 25, 8, 12 in this order is

sol:

the AVL tree after inserting the elements is 15

```

      /  \
    10  20
   /  \  \
  8 12 25

```

- is this tree balanced? yes for node 15, left: 3, right: 3 for node 20, left: 0, right: 1
- for post order traversal, we visit the left subtree, then the right subtree when entering the subtree, we will visit the left subtree (or node) first, then the right subtree (or node) and finally the root node of two subtrees; sequence: $8 \rightarrow 12 \rightarrow 10 \rightarrow 25 \rightarrow 20 \rightarrow 15$
- for preorder traversal, we visit the root node first, then the left node and its tree, then right node and its tree: $15 \rightarrow 10 \rightarrow 8 \rightarrow 12 \rightarrow 20 \rightarrow 25$
- inorder traversal: left subtree $\rightarrow$ root node $\rightarrow$ right subtree $8 \rightarrow 10 \rightarrow 12 \rightarrow 15 \rightarrow 20 \rightarrow 25$, the

inorder traversal of binary AVL tree is always sorted.

28:

The following code uses probing to resolve collisions in a hash table.

```
int index = key % table.length;
while (table[index] != null) {
    index = (index + 1) % table.length; // linear probing
    table[index] = key;
```

sol:

TODO: REVIEW THIS

```
int index = key % table.length;
while (table[index] != null) {
    index = (index + 1) % table.length; // linear probing
    table[index] = key;
```

29.

What EDI principle is demonstrated by using accessible color schemes in a Java Swing GUI, ensuring that users with visual impairments are not excluded?

sol:

Inclusion

30.

What EDI principle is reflected when a Java app project is designed to accommodate both younger and older users, screen reader users, and speakers of different languages?

sol:

Inclusion

Part B. Coding Questions

Question 1

Scorable Quiz System Given the following interface:

```
public interface Scorable {
    int getScore();
}
```

Complete the class QuizResult that implements Scorable. The getScore() method should return the percentage score as an integer (e.g., 80 for 8/10). If there are no questions, simply return 0. Use standard rounding (i.e., round to the nearest whole number). Starter Code:

```
public class QuizResult implements Scorable {
    private int correctAnswers;
    private int totalQuestions;
    public QuizResult(int correctAnswers, int totalQuestions) { // TO DO
    }
    // TO DO
}
```

sol:

```
public class QuizResult implements Scorable {
    private int correctAnswers;
    private int totalQuestions;

    public QuizResult(int correctAnswers, int totalQuestions) {
        this.correctAnswers = correctAnswers;
        this.totalQuestions = totalQuestions;
    }

    public int getScore() {
        if (totalQuestions == 0)
            return 0;
        return Math.round(correctAnswers / totalQuestions * 100);
    }
}
```

Question 2

You are writing a Java program that implements an airplane ticket. The ticket can be compared by passenger name, ticket price, arrival time, and seat class in that order.

Moreover, the seat class is implemented using an/enumeration with First Class, Business, and Economy, compared in that order.

Your task is to complete the TicketComparator class.

Date and Comparator have been imported. You are not allowed to import other classes. The skeleton code is given below:

```
import java.util.Date;
import java.util.Comparator;

public class AirplaneTicket {
    private String passengerName;
    private int ticketPrice;
    private String flightNumber;
    private Date arrivalTime;
    private SeatClass seatClass;

    public enum SeatClass {
        FIRST_CLASS, BUSINESS, ECONOMY
    };
}
```

```

public AirplaneTicket(String passengerName, int ticketPrice, String
    ↪ flightNumber, Date arrivalTime,
        SeatClass seatClass) {
    this.passengerName = passengerName;
    this.ticketPrice = ticketPrice;
    this.flightNumber = flightNumber;
    this.arrivalTime = arrivalTime;
    this.seatClass = seatClass;
}

public String getPassengerName() {
    return passengerName;
}

public int getTicketPrice() {
    return ticketPrice;
}

public String getFlightNumber() {
    return flightNumber;
}

public Date getArrivalTime() {
    return arrivalTime;
}

public SeatClass getSeatClass() {
    return seatClass;
}

public static Comparator<AirplaneTicket> getTicketComparator() {
    return new TicketComparator();
}

// Complete the class TicketComparator
private static class TicketComparator implements Comparator<AirplaneTicket> {

}
}

```

sol:

```

import java.util.Date;
import java.util.Comparator;

public class AirplaneTicket {
    private String passengerName;
    private int ticketPrice;
    private String flightNumber;
    private Date arrivalTime;
    private SeatClass seatClass;

    public enum SeatClass {
        FIRST_CLASS, BUSINESS, ECONOMY
    };
}

```

```
public AirplaneTicket(String passengerName,
    int ticketPrice,
    String flightNumber,
    Date arrivalTime,
    SeatClass seatClass) {
    this.passengerName = passengerName;
    this.ticketPrice = ticketPrice;
    this.flightNumber = flightNumber;
    this.arrivalTime = arrivalTime;
    this.seatClass = seatClass;
}

public String getPassengerName() {
    return passengerName;
}

public int getTicketPrice() {
    return ticketPrice;
}

public String getFlightNumber() {
    return flightNumber;
}

public Date getArrivalTime() {
    return arrivalTime;
}

public SeatClass getSeatClass() {
    return seatClass;
}

public static Comparator<AirplaneTicket> getTicketComparator() {
    return new TicketComparator();
}

// Complete the class TicketComparator
private static class TicketComparator implements Comparator<AirplaneTicket> {
    @Override
    public int compare(AirplaneTicket t1, AirplaneTicket t2) {
        // Compare by passenger name first
        int nameComp = t1.getPassengerName().compareTo(t2.getPassengerName());
        if (nameComp != 0) {
            return nameComp;
        }
        // then ticket price
        int priceComp = Integer.compare(t1.getTicketPrice(), t2.getTicketPrice());
        if (priceComp != 0) {
            return priceComp;
        }
        // then arrival time
        int arrivalComp = t1.getArrivalTime().compareTo(t2.getArrivalTime());
        if (arrivalComp != 0) {
            return arrivalComp;
        }
        // then seta class
        // we can convert the enum to integer for comparison
    }
}
```

```
        return Integer.compare(t1.getSeatClass().ordinal(),  
                               ↪ t2.getSeatClass().ordinal());  
    }  
}
```